

Disciplina de Sistemas Operacionais

Simulador Didático de Sistemas Operacionais

Relatório técnico

Anthony Kewin Pinto	RA 115738
Isadora Vischi da Cruz	RA 114640
Lucas Gabriel Risso	RA 114544
Victor Tazoi	RA 114923
Yan Vinícius Pimenta Silva	RA 115273

Professor: Maurício Acconcia Dias

Repositório: <https://github.com/grupo-fho/simulador-so>

Versão documentada: Java 21

20 de setembro de 2026

Objetivo do projeto

O trabalho consiste em um simulador de mecanismos de sistemas operacionais. A proposta não é substituir o sistema operacional da máquina, mas representar de forma controlada situações estudadas na disciplina: criação e execução de processos e threads, escalonamento da CPU, paginação, operações de entrada e saída e acesso a um sistema de arquivos simplificado.

O tempo é representado por um clock lógico. Dessa forma, o resultado não depende da velocidade do computador em que o programa é executado. Uma mesma carga, com os mesmos parâmetros, produz a mesma sequência de eventos e permite comparar políticas sob as mesmas condições.

A versão atual oferece uma CLI, uma interface gráfica em Swing e geração de logs e arquivos CSV. A interface gráfica é uma camada adicional; o núcleo utilizado por ela é o mesmo da execução por linha de comando.

Arquitetura

A implementação foi dividida por responsabilidade. O núcleo da simulação fica em **Simulador**, enquanto as regras de CPU, memória, E/S e arquivos ficam em classes próprias. Essa separação facilita a leitura e evita que toda a lógica fique concentrada em uma única classe.

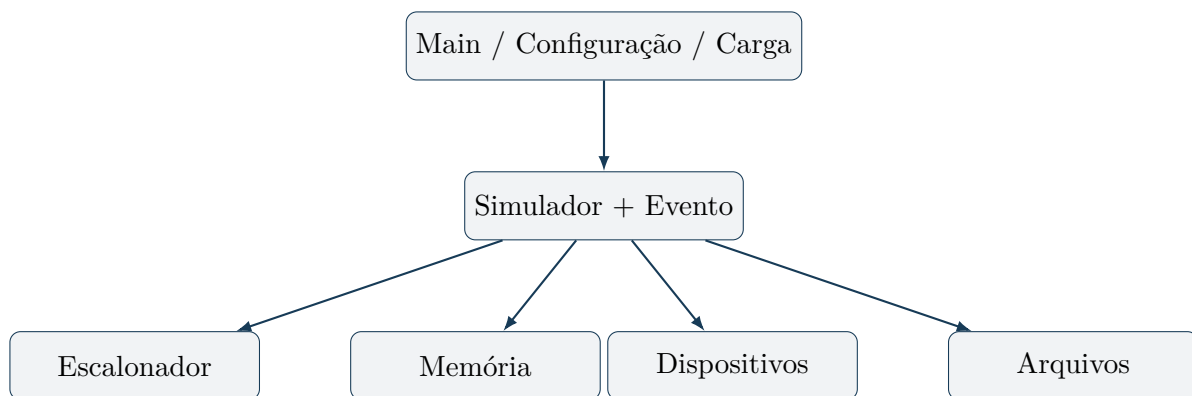


Figura 1: Organização geral do simulador.

Processo representa as informações equivalentes ao PCB. A classe mantém identificador, prioridade, chegada, estado agregado, threads, espaço de endereçamento e descritores de arquivos. **ThreadSimulada** representa o TCB e armazena estado, contador de programa, registradores simulados, pilha lógica e dados usados nas métricas.

O diagrama de classes completo está em `docs/diagrama-classes.mmd`.

Clock, eventos e estados

O simulador avança o clock até o próximo evento agendado. Quando mais de um evento ocorre no mesmo instante, a ordem adotada é: conclusão de E/S ou paginação, chegada, término de unidade da CPU e fim de troca de contexto. Empates dentro do mesmo tipo seguem a sequência de inserção.

As threads percorrem os estados NOVO, PRONTO, EXECUTANDO, BLOQUEADO e FINALIZADO. Cada mudança é registrada no log com a causa que provocou a transição. Por exemplo, uma falta de página leva a thread de EXECUTANDO para BLOQUEADO; quando a página é carregada, ela volta para PRONTO.

O estado do processo é obtido a partir das suas threads e das operações pendentes. Por isso, o PCB não precisa repetir exatamente todas as transições do TCB.

Escalonamento da CPU

Foram implementadas as três políticas obrigatórias: FCFS, Round Robin e Prioridade. No FCFS, a thread permanece na CPU até bloquear ou terminar. No Round Robin, o quantum limita o tempo contínuo de execução; se o surto não terminar, a thread retorna ao fim da fila. Na política de Prioridade, o menor valor numérico representa maior prioridade.

A política de Prioridade é não preemptiva. Portanto, a chegada de uma thread com prioridade melhor não interrompe imediatamente a que já está executando. A nova thread será considerada no próximo despacho.

A troca de contexto possui custo configurável. Esse intervalo entra no clock total, mas é separado do tempo de CPU útil. O simulador também conta quantas trocas foram realizadas.

Gerenciamento de memória

A memória usa paginação com páginas e molduras de tamanho fixo. Cada processo possui sua própria tabela de páginas. Um endereço lógico é dividido em número da página e deslocamento; quando a página está presente, o endereço físico é calculado pela moldura associada.

Se a página não estiver residente, ocorre uma falta de página. A thread é bloqueada e o pedido entra na fila do paginador. Havendo moldura livre, ela é utilizada. Caso contrário, a vítima é escolhida por FIFO. Depois do tempo de atendimento, a tabela de páginas é atualizada e a thread volta para a fila de prontas.

A referência que gerou a falta é concluída no evento de paginação. Isso evita registrar a mesma referência duas vezes quando a thread retoma a execução.

Entrada e saída

O simulador possui um dispositivo de bloco e um de caractere, cada um com fila FCFS e tempo de serviço próprio. Uma operação bloqueante retira a thread da CPU até a conclusão. A conclusão gera uma interrupção simulada e devolve a thread ao estado PRONTO.

Também existe operação não bloqueante. Nesse caso, a thread continua executando e o resultado é entregue posteriormente pela caixa de entrada do TCB. Não é utilizada espera ativa.

Sistema de arquivos

O sistema de arquivos é hierárquico e inicia em um diretório raiz. Arquivos e diretórios possuem metadados simplificados, como nome, tipo, permissões, tamanho e instantes lógicos de criação e modificação.

Foram implementadas as operações de criar, abrir, ler, escrever, fechar, remover e listar. Cada processo possui uma tabela local de descritores, ligada à tabela global de arquivos abertos.

Os dados dos arquivos usam alocação encadeada em blocos. Essa estratégia não exige blocos contíguos, mas o acesso a posições mais distantes depende do percurso da cadeia. O volume é mantido apenas durante a execução e não substitui o sistema de arquivos real do computador.

Interface e execução

A carga de trabalho fica fora do código-fonte. O arquivo descreve processos, chegadas, prioridades, threads e instruções. Parâmetros de escalonamento, memória e dispositivos podem ser informados pela CLI ou pela interface gráfica.

A GUI foi feita em Swing. Ela utiliza `Main.executar`, o mesmo fluxo da linha de comando, e roda a simulação em `SwingWorker` para manter a janela responsiva. O uso de `SwingWorker` não cria paralelismo entre as threads simuladas; o núcleo continua orientado a eventos e controlado pelo clock lógico.

Métricas

As métricas são calculadas a partir do clock lógico. As definições utilizadas são:

- **Tempo de retorno:** finalização menos chegada;
- **Tempo de espera:** soma dos intervalos na fila de prontas;
- **Tempo de resposta:** chegada até a primeira utilização da CPU;
- **Utilização da CPU:** unidades úteis divididas pelo tempo total;
- **Utilização dos dispositivos:** tempo ocupado dividido pelo tempo total;
- **Throughput:** processos concluídos por unidade de tempo;
- **Taxa de faltas:** faltas de página divididas pelo número de referências;
- **Trocas de contexto:** quantidade de trocas e sobrecarga acumulada.

Experimentos

Os experimentos foram executados com cargas controladas. Nas comparações, os parâmetros que não estavam sendo estudados foram mantidos constantes.

Comparação das políticas de CPU

A primeira comparação usa a mesma carga mista para FCFS, Round Robin com quantum 3 e Prioridade.

Política	Total	Trocas	Resposta média	Espera média	Retorno médio
FCFS	70	13	17,75	30,25	63,33
RR, q=3	87	24	6,25	42,00	69,00
Prioridade	75	12	15,00	28,75	50,33

Tabela 1: Comparação das políticas de escalonamento.

Nesse cenário, o Round Robin oferece a menor resposta média, pois distribui a CPU mais cedo entre as threads. Em contrapartida, realiza mais trocas de contexto e aumenta o tempo total. A política de Prioridade apresenta o menor retorno médio entre os três resultados, enquanto o FCFS termina o conjunto em menos tempo que o RR. As métricas mostram que cada política favorece um aspecto diferente da execução.

A quantidade de faltas também muda entre as políticas, mesmo com a configuração de memória igual. Isso ocorre porque o escalonamento altera a ordem em que as referências são feitas.

Efeito do quantum

O segundo experimento usa Round Robin e altera apenas o quantum.

Quantum	Total	Trocas	Resposta média	CPU útil
1	122	54	3,25	44,26%
3	87	24	6,25	62,07%
6	76	15	10,00	71,05%

Tabela 2: Efeito do quantum no Round Robin.

O quantum 1 reduz o tempo de resposta, mas causa 54 trocas para as mesmas 54 unidades de CPU útil. O quantum 6 reduz a sobrecarga e o tempo total, mas demora mais para oferecer a primeira oportunidade de CPU. O resultado mostra o compromisso entre responsividade e custo de troca.

Quantidade de molduras

A terceira comparação usa uma carga de memória que repete três páginas e altera somente a quantidade de molduras.

Molduras	Faltas	Taxa de faltas	Total	Ociosos
2	9	100,00%	65	45
3	3	33,33%	29	15

Tabela 3: Comparação da quantidade de molduras.

Com três molduras, as três primeiras referências carregam as páginas e as seis seguintes encontram seus dados residentes. Com duas molduras, a sequência provoca substituições sucessivas. Nessa carga, aumentar uma moldura reduz de forma clara os bloqueios e o tempo total.

Testes e validação

A versão atual foi validada com Java 21. A suíte registra 191 verificações aprovadas e cobre regras do núcleo, cenários de integração, interface e validações do lançador do DOOM. Entre os pontos verificados estão transições de estado, filas de escalonamento, quantum, métricas, tradução de endereços, FIFO, E/S bloqueante e não bloqueante, descritores e operações de arquivo.

Além dos testes automatizados, o repositório possui cargas pequenas que permitem conferir manualmente a ordem dos eventos. Essa conferência é útil para demonstrar que o resultado do simulador pode ser acompanhado passo a passo.

Antes da entrega, a distribuição deve ser testada novamente em uma cópia limpa, seguindo apenas as instruções do README.

Limitações

O simulador trabalha com uma única CPU lógica e tempos de serviço fixos. Não foram implementados multiprocessamento real, TLB, páginas sujas, envelhecimento de prioridade, deadlocks, semáforos, cache de blocos, journaling ou persistência física do volume simulado.

O paginador possui seu próprio atendimento e não converte automaticamente uma falta de página em requisição ao objeto que representa o disco. Da mesma forma, as operações do sistema de arquivos não são transformadas em E/S do dispositivo de bloco. Essas escolhas simplificam o modelo e devem ser consideradas ao interpretar os resultados.

A integração com Chocolate Doom também é separada do núcleo. O jogo é iniciado por `ProcessBuilder` como processo real do sistema hospedeiro e não é escalonado pelo simulador.

Conclusão

A implementação integra os mecanismos principais pedidos no trabalho em um mesmo eixo de tempo lógico. O uso de logs e métricas permite acompanhar as decisões do simulador e comparar configurações sem depender do tempo real da máquina.

Os experimentos mostram três efeitos importantes: a política de CPU altera resposta, espera e sobrecarga; o quantum do Round Robin modifica diretamente o número de trocas; e a quantidade de molduras influencia as faltas de página e o tempo de execução. Esses resultados reforçam o objetivo didático do projeto, que é observar como decisões de um sistema operacional afetam o comportamento da carga.

Ferramentas e materiais

O desenvolvimento e a validação utilizam JDK 21, scripts PowerShell, Python para automação dos experimentos e Git/GitHub para versionamento. O enunciado da disciplina é a referência para o escopo e os critérios de entrega. Ferramentas auxiliares adicionais que tenham sido utilizadas pelo grupo devem permanecer declaradas no README conforme as regras acadêmicas da disciplina.